Support Vector Machine



Математические методы распознавания образов, ВМК МГУ

Оглавление

1 Введение . . . 1
2 Случай линейно разделимой выборки . . . 2
2.1 Линейный классификатор и отступ . . . 2
2.2 Постановка задачи и прямая (primal) задача . . . 3
2.3 Переход к двойственной (dual) задаче . . . 3
2.4 Опорные векторы . . . 5
3 Случай линейно неразделимой выборки . . . 6
3.1 Восстановление решения прямой задачи . . . 7
3.2 Безусловная постановка и hinge loss . . . 8
4 Kernel trick и спрямляющее пространство . . . 10
4.1 Использование ядер на практике . . . 14
5 Как обучают SVM на практике . . . 17
5.1 Градиентный спуск по hinge loss . . . 17
5.2 Решение двойственной задачи: SMO . . . 19
5.3 Линейный случай: покоординатный подъем . . . 21
5.4 Ядра для больших выборок: явное ядровое отображение . . . 21
5.5 Калибровка вероятностей . . . 23
6 Вариации Support Vector Machine . . . 23
6.1 Многоклассовый SVC . . . 23
6.2 SVM для задач регрессии (SVR) . . . 25
6.3 SVM для обнаружения выбросов (одноклассовый SVM) . . . 30

Авторы: Афанасьев Глеб, Богачев Владимир, Кожемякова Елизавета

1 Введение

Рассмотрим задачу бинарной классификации. Нам дана выборка $(x^{(i)}, y^{(i)})_{i=1}^{l}$, метки классов договоримся обозначать $+1$ и -1.

Вы уже успели познакомиться с моделью логистической регрессии:

$$\mathbb{P}(y \mid x, w, b) = \sigma\big(y \cdot (\langle w, x\rangle + b)\big)$$
$$L(w, b) := \sum_{i=1}^{l} \log\Big(1 + \exp\big(-y^{(i)} \cdot (\langle w, x^{(i)}\rangle + b)\big)\Big) \to \min_{w,b}$$

Логистическая регрессия обучается на задачу правильного предсказания распределения меток класса объекта $y \mid x$. Теперь рассмотрим два правильно классифицированных объекта:

$$\mathbb{P}(y^{(1)} \mid x^{(1)}, w, b) = 0.9 \qquad \mathbb{P}(y^{(2)} \mid x^{(2)}, w, b) = 0.999.$$

Несмотря на правильную классификацию, потери на этих объектах существенно отличаются. Потери объекта 1 составляют $-\log(0.9) \approx 0.10536$, потери объекта 2 составляют $-\log(0.999) \approx 0.00100$.

Теперь рассмотрим случай дисбаланса классов. Пусть линейно разделимая обучающая выборка состоит из 3 объектов класса $+1$ и 1000 объектов класса -1. Так как при обучении LogReg максимизирует уверенность правильной классификации, то модели будет выгодно классифицировать объекты многочисленного класса -1 более уверенно, чем объекты малочисленного класса $+1$. Пример разделяющей гиперплоскости (прямой) можно увидеть на рисунке 1.

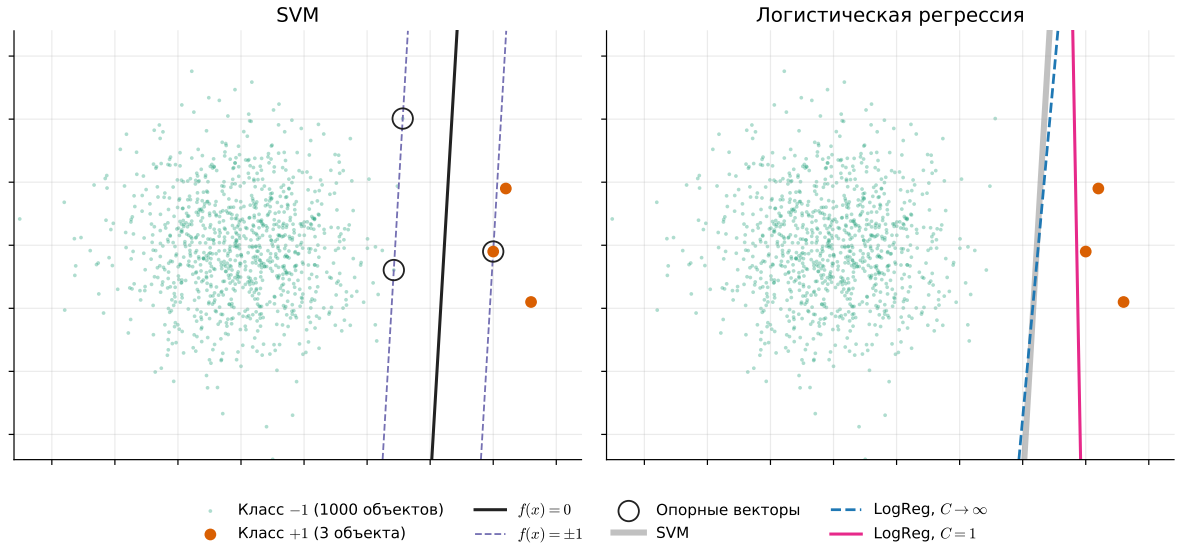


Рис. 1. Разделяющие прямые SVM и логистической регрессии при дисбалансе классов.

Поэтому LogReg с невысоким коэффициентом регуляризации может иметь плохую

обобщающую способность для класса $+1$.

Отметим, что на линейно разделимой выборке логистическую регрессию *необходимо* регуляризовать. Пусть (w, b) разделяет выборку, то есть $y^{(i)}(\langle w, x^{(i)}\rangle + b) > 0$ для всех i. Тогда при замене $(w, b) \to (tw, tb)$ каждое слагаемое $L(tw, tb)$ монотонно убывает к нулю при $t \to +\infty$. Значит, $\inf L = 0$, но минимум не достигается, и без регуляризации градиентный спуск будет неограниченно увеличивать $\|w\|$, продолжая уменьшать функцию потерь. Поэтому на рисунке 1 используется логистическая регрессия с L_2-регуляризацией.

Аналогичная проблема наблюдается и для выбросов. Если шумовой объект окажется глубоко в области чужого класса, то его вклад в функционал $L(w, b)$, равный $\log\bigl(1 + \exp(-y^{(i)}(\langle w, x^{(i)}\rangle + b))\bigr)$, растет почти линейно по мере удаления объекта от границы вглубь чужого класса. Поскольку потери суммируются по всем объектам, один выброс может заметно сместить разделяющую гиперплоскость и ухудшить классификацию остальных, «хороших» объектов.

Хотелось бы иметь метод, в котором положение гиперплоскости определяется лишь небольшим числом объектов вблизи границы, а влияние выбросов ограничено. Такой метод называется методом опорных векторов (SVM), и далее мы его построим.

2 Случай линейно разделимой выборки

2.1 Линейный классификатор и отступ

Рассмотрим линейный классификатор. Для чистоты рассуждений предположим, что выборка линейно разделима.

$$h_{w,b}(x) = \begin{cases} +1, & \langle w, x\rangle + b \geqslant 0, \\ -1, & \langle w, x\rangle + b < 0, \end{cases}$$

где w— веса классификатора, b— сдвиг.

Нетрудно заметить, что $\langle w, x\rangle + b = 0$ является уравнением гиперплоскости в признаковом пространстве с нормалью w и сдвигом b. Кроме того, это же уравнение определяет геометрическое место точек, в классификации которых алгоритм максимально не уверен.

Соответственно, при наличии обучающей выборки возникает естественное желание найти такие w и b, чтобы разделяющая гиперплоскость лежала максимально далеко от разделяемых множеств.

Как вы помните из курса линейной алгебры, расстояние от точки $x^{(i)}$ до гиперплоскости равно

$$\frac{|\langle w, x^{(i)}\rangle + b|}{\|w\|}.$$

Так как метки принимают значения $y^{(i)} \in \{-1, +1\}$, а для правильно классифицированного объекта знак $\langle w, x^{(i)} \rangle + b$ совпадает с $y^{(i)}$, модуль можно заменить умножением на метку:

$$M^{(i)} := y^{(i)} \frac{\langle w, x^{(i)} \rangle + b}{\|w\|}.$$

Определение 2.1: Отступ

Число $M^{(i)}$ называется **отступом** (margin) объекта $x^{(i)}$. Отступ положителен, если объект классифицирован правильно, и отрицателен, если неправильно.

2.2 Постановка задачи и прямая (primal) задача

Задача поиска гиперплоскости, лежащей максимально далеко от обоих классов, записывается как максимизация минимального отступа $M_{\min} := \min\limits_i M^{(i)}$:

$$\begin{aligned} M_{\min} &\longrightarrow \max_{w,b}, \\ \texttt{s.t.} \quad M^{(i)} &\geqslant M_{\min}, \quad i = 1, \ldots, l. \end{aligned}$$

Если умножить w и b на положительную константу, то гиперплоскость и отступы $M^{(i)}$ не изменятся. Пользуясь этим, нормируем w и b так, что

$$\min_i y^{(i)} \big(\langle w, x^{(i)} \rangle + b \big) = 1.$$

Тогда $M_{\min} = \dfrac{1}{\|w\|}$, и максимизация $M_{\min}$ эквивалентна минимизации $\frac{1}{2}\|w\|^2$. Получаем задачу в линейно разделимом случае:

$$\begin{aligned} \frac{1}{2}\|w\|^2 &\longrightarrow \min_{w,b}, \\ \texttt{s.t.} \quad y^{(i)} \big(\langle w, x^{(i)} \rangle + b \big) &\geqslant 1, \quad i = 1, \ldots, l. \end{aligned} \tag{1}$$

Таким образом, мы пришли к выпуклой задаче условной оптимизации.

2.3 Переход к двойственной (dual) задаче

На позапрошлом семинаре вы рассматривали метод решения задач условной оптимизации, а именно условия ККТ. Применим этот метод к задаче (1).

2.3.1 Лагранжиан и условия стационарности

Запишем ограничения в виде

$$c_i(w, b) = 1 - y^{(i)} \big(\langle w, x^{(i)} \rangle + b \big) \leqslant 0$$

и введем множители Лагранжа $\lambda_i \geqslant 0$. Лагранжиан имеет вид

$$\mathcal{L}(w, b, \lambda) = \frac{1}{2}\|w\|^2 - \sum_{i=1}^{l} \lambda_i \big[y^{(i)}\big(\langle w, x^{(i)}\rangle + b\big) - 1\big].$$

Двойственная функция есть $g(\lambda) = \inf_{w,b} \mathcal{L}(w, b, \lambda)$. Сгруппируем слагаемые лагранжиана:

$$\mathcal{L}(w, b, \lambda) = \underbrace{\frac{1}{2}\|w\|^2 - \sum_{i=1}^{l} \lambda_i y^{(i)} \langle w, x^{(i)}\rangle}_{\text{зависит только от } w} - b\sum_{i=1}^{l} \lambda_i y^{(i)} + \sum_{i=1}^{l} \lambda_i.$$

Переменные разделились, поэтому минимизировать по w и по b можно независимо.

По w лагранжиан — строго выпуклая квадратичная функция, и ее минимум находится из условия стационарности:

$$\nabla_w \mathcal{L} = w - \sum_{i=1}^{l} \lambda_i y^{(i)} x^{(i)} = 0 \quad \Longrightarrow \quad w = \sum_{i=1}^{l} \lambda_i y^{(i)} x^{(i)}. \tag{2}$$

По b лагранжиан *линеен* с коэффициентом $-\sum_i \lambda_i y^{(i)}$. Линейная функция ограничена снизу на всей прямой, только если ее коэффициент равен нулю: иначе, устремляя b к $+\infty$ или к $-\infty$, получаем $\inf_b \mathcal{L} = -\infty$. Поэтому

$$g(\lambda) = \begin{cases} \min\limits_{w}\Big(\dfrac{1}{2}\|w\|^2 - \sum\limits_{i=1}^{l} \lambda_i y^{(i)} \langle w, x^{(i)}\rangle\Big) + \sum\limits_{i=1}^{l} \lambda_i, & \text{если } \sum_{i=1}^{l} \lambda_i y^{(i)} = 0, \\ -\infty, & \text{иначе.} \end{cases}$$

Так как в методе множителей Лагранжа двойственная функция $g(\lambda)$ в конечном счете максимизируется по λ, точки, в которых $g(\lambda) = -\infty$, заведомо не являются решением, и их можно исключить. Тем самым мы получаем ограничение двойственной задачи

$$\sum_{i=1}^{l} \lambda_i y^{(i)} = 0. \tag{3}$$

Формально оно совпадает с условием $\partial\mathcal{L}/\partial b = 0$, но получено из других соображений: это не условие экстремума, а описание множества, на котором двойственная функция конечна.

2.3.2 Двойственная функция

Найдем $g(\lambda)$ при выполнении (3). Слагаемое с b в этом случае равно нулю. Обозначим

$$S := \sum_{i,j=1}^{l} \lambda_i \lambda_j y^{(i)} y^{(j)} \langle x^{(i)}, x^{(j)}\rangle.$$

Тогда, пользуясь (2), получаем

$$\|w\|^2 = \left\langle \sum_{i=1}^{l} \lambda_i y^{(i)} x^{(i)}, \sum_{j=1}^{l} \lambda_j y^{(j)} x^{(j)} \right\rangle = S,$$
$$\sum_{i=1}^{l} \lambda_i y^{(i)} \langle w, x^{(i)} \rangle = \left\langle w, \sum_{i=1}^{l} \lambda_i y^{(i)} x^{(i)} \right\rangle = \langle w, w \rangle = S.$$

Следовательно,

$$g(\lambda) = \frac{1}{2} S - S + \sum_{i=1}^{l} \lambda_i = \sum_{i=1}^{l} \lambda_i - \frac{1}{2} \sum_{i,j=1}^{l} \lambda_i \lambda_j y^{(i)} y^{(j)} \langle x^{(i)}, x^{(j)} \rangle.$$

С учетом ограничений $\lambda_i \geqslant 0$ и (3) двойственная задача имеет вид

$$\begin{aligned} \max_{\lambda} \quad & g(\lambda) = \sum_{i=1}^{l} \lambda_i - \frac{1}{2} \sum_{i,j=1}^{l} \lambda_i \lambda_j y^{(i)} y^{(j)} \langle x^{(i)}, x^{(j)} \rangle \\ \texttt{s.t.} \quad & \lambda_i \geqslant 0, \quad i = 1, \ldots, l, \\ & \sum_{i=1}^{l} \lambda_i y^{(i)} = 0. \end{aligned} \tag{4}$$

2.4 Опорные векторы

В условия ККТ также входят условия дополняющей нежесткости

$$\lambda_i \left[y^{(i)} \left(\langle w, x^{(i)} \rangle + b \right) - 1 \right] = 0, \quad i = 1, \ldots, l.$$

Они означают, что в решении двойственной задачи ненулевыми могут быть только те λ_i, которые соответствуют объектам с минимальным отступом.

Определение 2.2: Опорные векторы

Объекты выборки с минимальным отступом, которым соответствуют ненулевые λ_i, называются **опорными векторами**.

Решив двойственную задачу, веса w можно восстановить по формуле (2), при этом в сумме участвуют только опорные векторы. Чтобы найти b, возьмем любой опорный вектор $x^{(i)}$. Для него $\lambda_i > 0$, поэтому по условию дополняющей нежесткости

$$y^{(i)} \left(\langle w, x^{(i)} \rangle + b \right) = 1.$$

Так как $y^{(i)} \in \{-1, +1\}$, получаем

$$b = y^{(i)} - \langle w, x^{(i)} \rangle = y^{(i)} - \sum_{j=1}^{l} \lambda_j y^{(j)} \langle x^{(j)}, x^{(i)} \rangle.$$

В точном решении любой опорный вектор дает одно и то же значение b.

3 Случай линейно неразделимой выборки

Теперь перейдем к более общему случаю, когда выборку невозможно разделить гиперплоскостью. Это означает, что какие бы w и b мы ни взяли, хотя бы одно из ограничений в (1) будет нарушено, и, следовательно, задача неразрешима. Переформулируем ее следующим образом:

$$\begin{aligned} & \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{l} \xi_i \longrightarrow \min_{w,b,\xi}, \\ \texttt{s.t.} \quad & y^{(i)}\big(\langle w, x^{(i)} \rangle + b\big) \geqslant 1 - \xi_i, \quad i = 1, \ldots, l, \\ & \xi_i \geqslant 0, \quad i = 1, \ldots, l. \end{aligned} \tag{5}$$

Таким образом, мы допускаем, что ряд элементов обучающей выборки может оказаться не в «своей» части пространства, но требуем от алгоритма штрафовать такие элементы. Параметр C определяет величину штрафа.

Введем множители Лагранжа $\lambda_i \geqslant 0$ для первой группы ограничений и $\mu_i \geqslant 0$ для ограничений $\xi_i \geqslant 0$:

$$\mathcal{L}(w, b, \xi, \lambda, \mu) = \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{l} \xi_i - \sum_{i=1}^{l} \lambda_i \big[y^{(i)}\big(\langle w, x^{(i)} \rangle + b\big) - 1 + \xi_i\big] - \sum_{i=1}^{l} \mu_i \xi_i.$$

По w и b все рассуждения раздела 2 повторяются дословно: получаем ту же формулу (2) для w и то же ограничение (3). По каждому ξ_i лагранжиан тоже линеен, с коэффициентом $C - \lambda_i - \mu_i$, поэтому двойственная функция конечна, только если

$$\mu_i = C - \lambda_i, \quad i = 1, \ldots, l.$$

Тогда все слагаемые с ξ_i сокращаются, а условие $\mu_i \geqslant 0$ превращается в $\lambda_i \leqslant C$. Получаем двойственную задачу следующего вида:

$$\begin{aligned} \max_{\lambda} \quad & g(\lambda) = \sum_{i=1}^{l} \lambda_i - \frac{1}{2}\sum_{i,j=1}^{l} y^{(i)} y^{(j)} \lambda_i \lambda_j \langle x^{(i)}, x^{(j)} \rangle \\ \texttt{s.t.} \quad & 0 \leqslant \lambda_i \leqslant C, \quad i = 1, \ldots, l, \\ & \sum_{i=1}^{l} \lambda_i y^{(i)} = 0. \end{aligned} \tag{6}$$

Замечание 3.1

Единственное изменение по сравнению с (4)— верхняя граница C на множители Лагранжа.

3.1 Восстановление решения прямой задачи

Пусть λ— решение двойственной задачи (6). Веса по-прежнему восстанавливаются по формуле (2): $w = \sum_i \lambda_i y^{(i)} x^{(i)}$. Со сдвигом b ситуация сложнее, и для него нам понадобятся условия дополняющей нежесткости

$$\lambda_i \big[m^{(i)} - 1 + \xi_i \big] = 0, \qquad (C - \lambda_i)\, \xi_i = 0, \quad i = 1, \ldots, l,$$

где $m^{(i)} := y^{(i)} \big(\langle w, x^{(i)} \rangle + b \big)$— отступ без нормировки на $\|w\|$. Из них получаем три типа объектов:

- $\lambda_i = 0$: тогда $\xi_i = 0$ и $m^{(i)} \geqslant 1$— объект лежит на своей стороне вне разделяющей полосы или на ее границе;
- $0 < \lambda_i < C$: тогда $\xi_i = 0$ и $m^{(i)} = 1$— объект лежит ровно на границе полосы;
- $\lambda_i = C$: тогда $m^{(i)} = 1 - \xi_i \leqslant 1$— объект лежит на границе полосы, внутри нее или на чужой стороне.

Опорными векторами называются объекты с $\lambda_i > 0$; все они имеют $m^{(i)} \leqslant 1$. Обратно, любой объект с $m^{(i)} < 1$ является опорным, причем для него $\lambda_i = C$.

Невырожденный случай. Если найдется хотя бы один объект с $0 < \lambda_i < C$, то для него $m^{(i)} = 1$ и, как в разделимом случае,

$$b = y^{(i)} - \langle w, x^{(i)} \rangle.$$

Вырожденный случай. Может оказаться, что все λ_i равны либо 0, либо C. Это вполне типично при малых C: штраф за нарушение полосы мал, полоса широкая, и почти все опорные векторы оказываются внутри нее. Тогда равенств для b нет, но условия ККТ дают неравенства. Обозначим $b_i := y^{(i)} - \langle w, x^{(i)} \rangle$— значение сдвига, при котором объект i лежит ровно на границе полосы. Так как $\big(y^{(i)}\big)^2 = 1$, имеем

$$m^{(i)} = 1 + y^{(i)}(b - b_i).$$

Условие $m^{(i)} \geqslant 1$ при $\lambda_i = 0$ и условие $m^{(i)} \leqslant 1$ при $\lambda_i = C$ дают

$$\max_{i \in I_{\text{low}}} b_i \;\leqslant\; b \;\leqslant\; \min_{i \in I_{\text{up}}} b_i,$$

где

$$I_{\text{low}} = \{i \colon \lambda_i = 0,\ y^{(i)} = +1\} \cup \{i \colon \lambda_i = C,\ y^{(i)} = -1\},$$
$$I_{\text{up}} = \{i \colon \lambda_i = 0,\ y^{(i)} = -1\} \cup \{i \colon \lambda_i = C,\ y^{(i)} = +1\}.$$

Существование решения прямой задачи гарантирует, что этот отрезок непуст. Любое b из него вместе с $\xi_i = \max(0, 1 - m^{(i)})$ удовлетворяет всем условиям ККТ, а значит, является оптимальным: в вырожденном случае сдвиг определен неоднозначно. Геометрически это означает, что гиперплоскость можно сдвигать вдоль w в некоторых пределах, не меняя значения функционала. В `libsvm` в этом случае берут середину отрезка.

Замечание 3.2

Если удалить из выборки все объекты с $\lambda_i = 0$ и заново обучить SVM, то исходная разделяющая гиперплоскость останется решением. Действительно, набор (w, b, ξ, λ), ограниченный на оставшиеся объекты, удовлетворяет условиям ККТ новой задачи: удаленные слагаемые в $w = \sum_i \lambda_i y^{(i)} x^{(i)}$ и в $\sum_i \lambda_i y^{(i)}$ были нулевыми, а остальные условия выполнены для каждого объекта по отдельности. Задача выпукла, и ограничения в ней линейны, поэтому условия ККТ достаточны для оптимальности. Иначе говоря, вся информация о решении содержится в опорных векторах.

Для логистической регрессии это не так: в ее решении каждый объект дает ненулевой вклад в градиент функции потерь, и удаление любого объекта, вообще говоря, меняет решение.

3.2 Безусловная постановка и hinge loss

Задачу (5) можно переписать без ограничений. Напомним, что

$$m^{(i)} = y^{(i)}\big(\langle w, x^{(i)}\rangle + b\big)$$

— отступ без нормировки на $\|w\|$, так что $M^{(i)} = m^{(i)}/\|w\|$. Зафиксируем w и b и посмотрим, какие ξ_i выгодно выбрать. Ограничения требуют $\xi_i \geqslant 1 - m^{(i)}$ и $\xi_i \geqslant 0$, а функционал возрастает по каждому ξ_i. Значит, в оптимуме ξ_i равно наименьшему допустимому значению:

$$\xi_i = \max\big(0, 1 - m^{(i)}\big).$$

Подставив это в (5) и поделив функционал на C, получаем эквивалентную задачу безусловной оптимизации:

$$\sum_{i=1}^{l} \max\Big(0, 1 - y^{(i)}\big(\langle w, x^{(i)}\rangle + b\big)\Big) + \frac{1}{2C}\|w\|^2 \longrightarrow \min_{w,b}. \tag{7}$$

Определение 3.3: Hinge loss

Функция

$$L_{\text{hinge}}(m) := \max(0, 1 - m) = (1 - m)_+$$

называется **кусочно-линейной функцией потерь** (hinge loss, «шарнирная функция потерь»).

График hinge loss изображен на рисунке 2. Ось абсцисс делится на три зоны: при $m < 0$ объект классифицирован неверно, при $0 \leqslant m < 1$ — верно, но попадает в разделяющую полосу, при $m \geqslant 1$ — верно и с запасом, и тогда потери равны нулю.

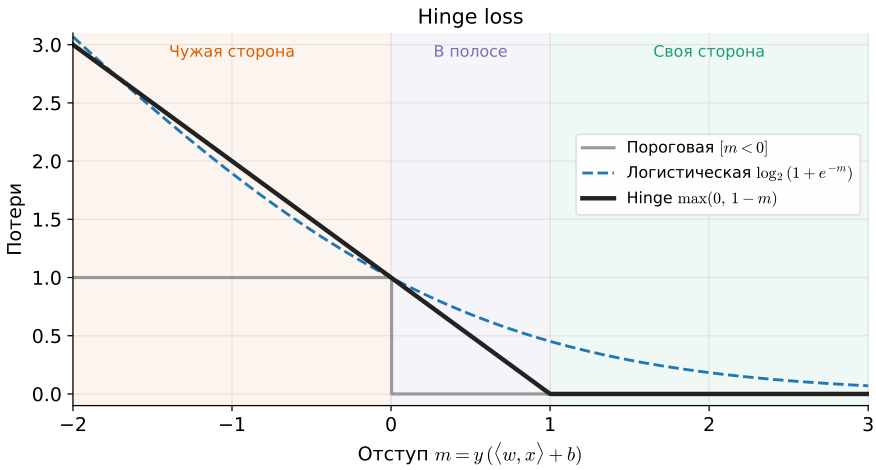


Рис. 2. Hinge loss (черная), пороговая функция потерь (серая) и логистическая функция потерь по основанию 2 (пунктир).

Таким образом, SVM — это линейный классификатор, обучаемый минимизацией эмпирического риска с функцией потерь hinge loss и L_2-регуляризацией. Параметр C обратен коэффициенту регуляризации: чем больше C, тем слабее регуляризация и тем сильнее штрафуются ошибки на обучающей выборке.

3.2.1 Сравнение с логистической регрессией

Логистическая регрессия с L_2-регуляризацией решает задачу того же вида:

$$\sum_{i=1}^{l} \log\left(1 + e^{-m^{(i)}}\right) + \frac{1}{2C}\|w\|^2 \longrightarrow \min_{w,b}.$$

Следовательно, SVM и логистическая регрессия отличаются *только функцией потерь* (рисунок 2). Это объясняет различия, отмеченные во введении:

- **Объекты с $m^{(i)} \geqslant 1$ не дают вклада.** Логистические потери положительны при любом отступе, поэтому модели всегда выгодно отодвигать гиперплоскость от объектов, даже если они уже классифицированы уверенно. При дисбалансе классов многочисленный класс «перетягивает» гиперплоскость на себя (рисунок 1). У hinge loss выигрыша от увеличения отступа сверх 1 нет, и положение гиперплоскости определяется только объектами с $m^{(i)} \leqslant 1$, то есть опорными векторами.

- **Разреженность решения.** По той же причине в формуле $w = \sum_i \lambda_i y^{(i)} x^{(i)}$ ненулевые коэффициенты есть только у опорных векторов. В логистической регрессии в аналогичном разложении участвуют все объекты. В частности, удаление неопорных объектов не меняет решения SVM (замечание 3.2), а у логистической регрессии такого свойства нет.
- **Ограниченное влияние одного объекта.** При $m^{(i)} < 1$ hinge loss растет линейно, поэтому ее производная по $m^{(i)}$ по модулю не больше 1, а множители Лагранжа ограничены: $\lambda_i \leqslant C$. Отметим, что логистические потери на хвосте тоже растут линейно, так что далекий выброс влияет на обе модели сопоставимо: полностью избавиться от его влияния можно только с невыпуклыми функциями потерь (например, ramp loss $\min\big(1, (1-m)_+\big)$).

На практике по качеству классификации оба метода обычно сопоставимы. Hinge loss ближе к пороговой функции потерь: она равна нулю у всех объектов с $m \geqslant 1$ и не тратит «усилия модели» на и без того уверенно классифицированные объекты. Поэтому SVM иногда дает небольшой выигрыш по `accuracy`, но это не гарантировано. Зато логистическая регрессия изначально выводилась как вероятностная модель $\mathbb{P}(y \mid x)$, и ее функция потерь (логарифмическая) является строго собственной оценкой вероятностей (strictly proper scoring rule). Поэтому логистическая регрессия «из коробки» выдает достаточно хорошо откалиброванные вероятности, а значение $f(x)$ у SVM вероятностью не является и требует дополнительной калибровки (раздел 5.5).

3.2.2 Связь с числом ошибок

Число ошибок классификатора на выборке равно $\sum_i [m^{(i)} < 0]$. Минимизировать его напрямую трудно: функция $[m < 0]$ разрывна и невыпукла. Поскольку

$$[m < 0] \leqslant \max(0, 1 - m) \quad \text{для всех } m,$$

hinge loss является *выпуклой верхней оценкой* пороговой функции потерь (на рисунке 2 черная линия всюду лежит не ниже серой). Поэтому задача (7) минимизирует (с регуляризацией) верхнюю оценку числа ошибок на обучающей выборке, а сверх этого штрафует еще и «неуверенно» классифицированные объекты с $0 \leqslant m^{(i)} < 1$.

Безусловная постановка (7) важна и практически: ее можно решать вариантом обычного градиентного спуска, не переходя к двойственной задаче. Подробнее об этом и о других способах обучения SVM — в разделе 5.

4 Kernel trick и спрямляющее пространство

Зачастую, данные в исходном признаковом пространстве не являются линейно разделимыми, но, преобразовав признаки, можно получить пространство, в котором выборка будет линейно разделимой (или близка к линейно разделимой). Пусть x — исходное признаковое описание, а $\tilde{x} = \mathcal{F}(x)$ — новое признаковое описание.

Рассмотрим пример перехода к спрямляющему пространству. Пусть признаковое описание $x \in \mathbb{R}^2$ двумерно, объекты класса $+1$ лежат в окрестности окружности радиуса $r_{+1} = 0.3$, объекты класса -1 лежат в окрестности окружности радиуса $r_{-1} = 1$ (выборка из 1000 объектов, к координатам добавлен гауссов шум со стандартным отклонением 0.025). На рисунке 3 продемонстрирован пример такой выборки и пример разделяющей кривой. Очевидно, что такая выборка не будет линейно разделимой и линейный классификатор не сможет показать достаточно высокую точность (`accuracy` у линейного SVM 0.69).

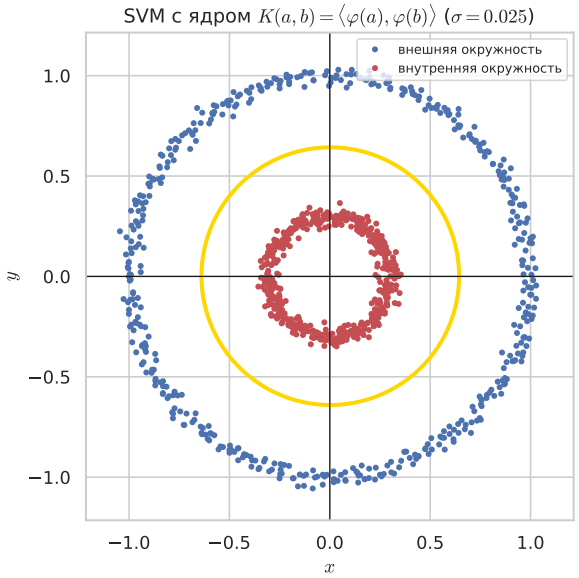


Рис. 3. Выборка из двух вложенных окружностей и разделяющая кривая SVM со стереографическим ядром (жёлтая линия).

Рассмотрим теперь пример перехода к спрямляющему пространству (рисунок 4), в котором данная выборка станет линейно разделимой. Рассмотрим обратную стереографическую проекцию, которая переводит точки плоскости на единичную сферу $\mathbb{S}^2 \subset \mathbb{R}^3$:

$$\mathcal{F}(x_1, x_2) = \frac{(2x_1,\ 2x_2,\ x_1^2 + x_2^2 - 1)}{x_1^2 + x_2^2 + 1}.$$

При подобном преобразовании окружности переходят в окружности, а концентрические началу координат окружности переходят в окружности параллельные $O\tilde{x}\,\tilde{y}$, а координаты $\tilde{z}$ точек конецентрической началу координат окружностей постоянны и зависят только от радиуса данной окружности ($\tilde{z} = \frac{r^2-1}{r^2+1}$). В нашем примере внутренняя окружность ($r = 0.3$) переходит в окружность на высоте $\tilde{z} \approx -0.84$, а внешняя ($r = 1$) — на экватор $\tilde{z} = 0$. В таком пространстве выборка становится линейно разделимой плоскостью $\tilde{z} = \texttt{Const}$ (например, $\tilde{z} = -0.4$). Линейный SVM, обученный в спрямляющем пространстве, показывает `accuracy` 1.00.

В данном случае мы смогли придумать способ преобразования признаков ($\tilde{x} = \mathcal{F}(x)$), который позволил получить линейно разделимую выборку. Однако, на практике та-

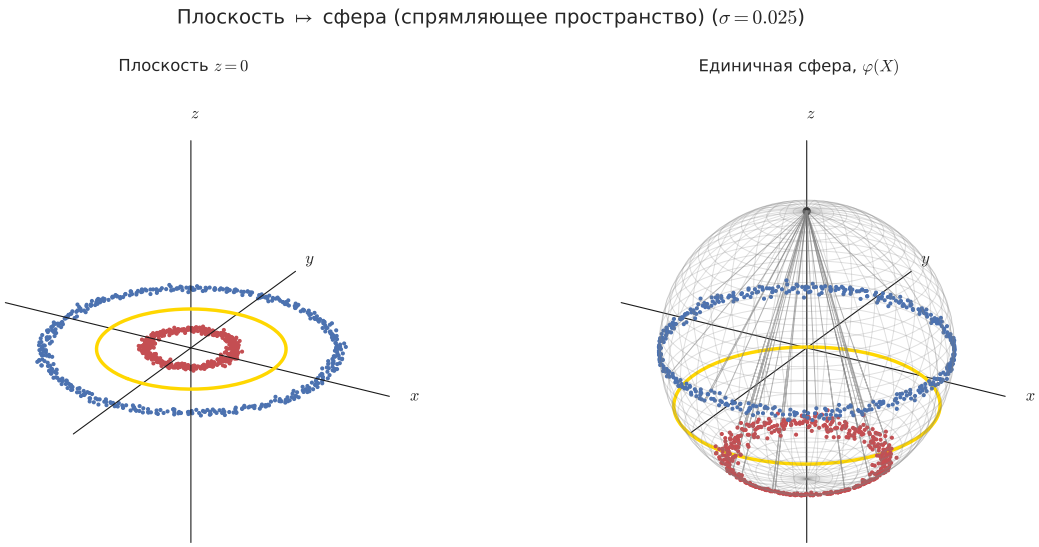


Рис. 4. Переход к спрямляющему пространству: плоскость $\tilde{z} = 0$ с исходной выборкой (слева) и единичная сфера с образом выборки $\mathcal{F}(X)$ (справа). Серые линии — лучи проекции из северного полюса.

кой подход оказывается затруднительным. Хотелось бы получить метод машинного обучениея, в котором поиск преобразования признаков автоматизирован.

Вернёмся к двойственной задаче обучения SVM. Можно заметить, что в двойственной задаче (в отличие от прямой) зависимость от объектов x_i только в попарных скалярных произведениях $\langle x_i, x_j \rangle$. Предположим, что мы уже выполнили преобразование $\tilde{x} = \mathcal{F}(x)$ и хотим обучить SVM в новом признаковом пространстве, тогда нам необходимо вычислять только $\langle \mathcal{F}(x_i), \mathcal{F}(x_j) \rangle$, а вычисление самого нового признакового описания $\tilde{x} = \mathcal{F}(x)$ не является обязательным. Вы уже знакомы с определением ядра:

Определение 4.1: Ядро (kernel)

Функция $K \colon \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ называется **ядром**, если существует отображение $\mathcal{F} \colon \mathcal{X} \to \mathcal{H}$ в некоторое евклидово или гильбертово пространство $\mathcal{H}$ (спрямляющее пространство) такое, что $K(x, z) = \langle \mathcal{F}(x), \mathcal{F}(z) \rangle$ для всех $x, z \in \mathcal{X}$.

Для двойственной задачи SVM это означает, что скалярные произведения $\langle x^{(i)}, x^{(j)} \rangle$ заменяются на $K(x^{(i)}, x^{(j)})$, а решающее правило принимает вид $f(x) = \sum_i \alpha_i y^{(i)} K(x^{(i)}, x) + b$.

На паре по ядрам вы познакомились со множеством примеров ядер, которые соответствуют различным преобразованиям признаков. Оказывается, что подобрать ядро $K(x, z)$ зачастую проще, чем само преобразование $\mathcal{F}(x)$.

Теперь вернёмся к иллюстрации 4. На рисунке изображен переход к спрямляющему пространству. Легко заметить, что в данном спрямляющем пространстве

выборка оказывается линейно разделимой. Но подобная визуализация возможна исключительно по двум причинам

- У нас есть доступ к преобразованию $\mathcal{F}(x)$, а не только к порождаемуму данным преобразованием ядру $K(x, z)$;
- Новое признаковое пространство $\tilde{x}$ оказывается трёхмерным, поэтому мы можем его визуализировать.

Данные ограничения не позволяют нам построить наглядную визуализацию перехода к спрямляющему пространству для более сложных задач.

Рассмотрим другой подход к визуализации: разложим спрямляющее пространство в прямую сумму $H \oplus H^{\perp}$, где H — разделяющая гиперплоскость, $H^{\perp} = \{\tilde{x} \colon \langle w, \tilde{x}\rangle = 0\}$ — нормаль к разделяющей гиперплоскости. Тогда score модели на объекте $\tilde{x}$ будет равен $\langle w, \tilde{x}\rangle + b$. Положим $n = \frac{w}{\|w\|}$, тогда score модели будет равен

$$\|w\|\big(\langle n, \tilde{x}\rangle + \frac{b}{\|w\|}\big) \;\propto\; \langle n, \tilde{x}\rangle + \frac{b}{\|w\|}.$$

Другими словами, проекция объекта $\tilde{x}$ на n (со сдвигом) пропорциональна score модели на данном объекте. Теперь рассмотрим выборку $\tilde{x}^{(i)}$ в спрямляющем пространстве, выполним ортогональную проекцию $\tilde{x} \mapsto \hat{x} \in H$ на разеделяющую гиперплоскость и применем к полученной выборке $\hat{x}^{(i)}$ PCA для выделения 1-й главной компоненты. Обозначим данную компоненту τ_1 (причем $\tau_1 \perp n$ по построению). Таким образом, мы получили ортонормированный базис двумерного подпространства спрямляющего пространства. Причем разделяющая гиперплоскость проецируется в прямую $\langle \tilde{x}, n\rangle = \text{Const}$. Осталось заметить, что координаты точки на получившейся плоскости

$$y' = \langle n, \tilde{x}\rangle + \frac{b}{\|w\|}, \qquad x' = \langle \tau_1, \tilde{x}\rangle,$$

где $w = \sum_i \alpha_i y^{(i)} \mathcal{F}(x^{(i)})$, $n = \frac{w}{\|w\|}$, а τ_1 — первая главная компонента выборки $\hat{x}^{(i)} = \tilde{x}^{(i)} - \langle n, \tilde{x}^{(i)}\rangle n$, зависят от $\tilde{x}$ только через скалярные произведения, поэтому мы можем перейти к аналогичным формулам

$$y' = \frac{f(x)}{\|w\|} = \frac{\sum_i \alpha_i y^{(i)} K(x^{(i)}, x) + b}{\|w\|}, \qquad \|w\|^2 = \sum_{i,j} \alpha_i \alpha_j y^{(i)} y^{(j)} K(x^{(i)}, x^{(j)}),$$

$$x' = \text{KPCA}_1(x) \;\text{ с ядром }\; \tilde{K}(x, z) = K(x, z) - k(x)\, k(z),$$

$$k(x) = \frac{\sum_i \alpha_i y^{(i)} K(x^{(i)}, x)}{\|w\|}$$

для исходного пространства (суммы берутся по опорным векторам). Здесь $k(x) = \langle \mathcal{F}(x), n\rangle$, а $\tilde{K}(x, z) = \langle \hat{x}, \hat{z}\rangle$ — скалярное произведение проекций $\mathcal{F}(x)$ и $\mathcal{F}(z)$ на разделяющую гиперплоскость. Знак компоненты KPCA определён с точностью до

разворота оси. Заметим, что y' — знаковое расстояние от объекта до разделяющей гиперплоскости в спрямляющем пространстве, поэтому гиперплоскость переходит в прямую $y' = 0$.

На рисунке 5 показан пример проекции спрямляющего пространства, полученного в примере выше (слева для сравнения — линейный SVM, $K(x, z) = \langle x, z \rangle$, справа — SVM со стереографическим ядром).

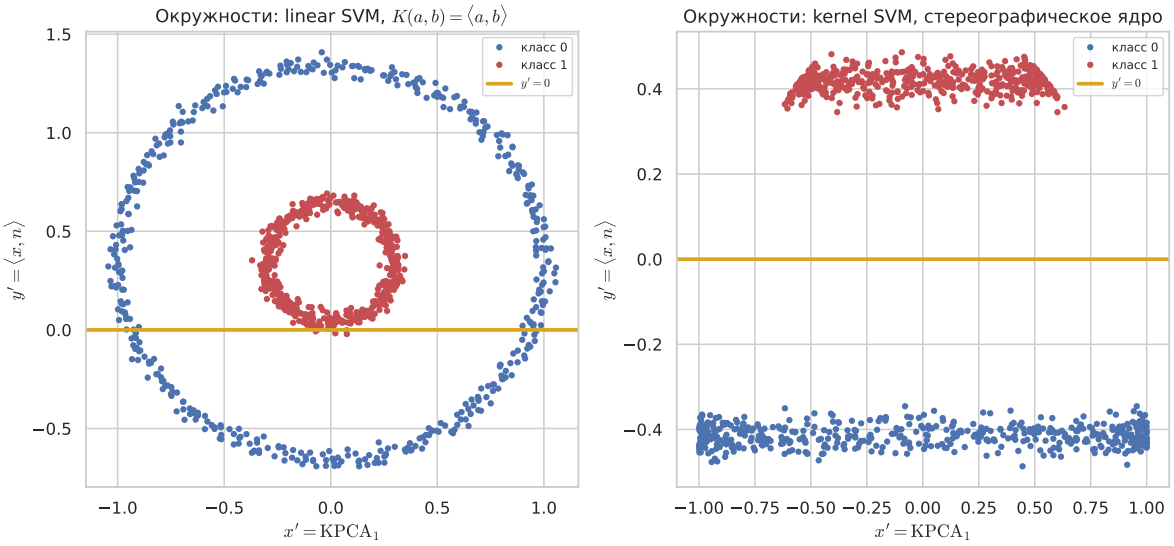


Рис. 5. Проекция (x', y') для вложенных окружностей: линейный SVM (слева) и SVM со стереографическим ядром (справа). Жёлтая прямая — образ разделяющей гиперплоскости.

На рисунке 6 показан пример проекции спрямляющего пространства, полученного для задачи бинарной классификации `Phoneme` (OpenML, 5 признаков, признаки стандартизованы, 2000 объектов в обучающей и 2000 в тестовой выборке, $C = 100$) применением ядра `RBF` с $\gamma = 1/(d \cdot \mathrm{Var}(X))$. Проекция построена по обучающей выборке, на рисунке изображены тестовые объекты. Тестовая `accuracy` линейного SVM равна 0.7775, а SVM с ядром `RBF` — 0.8555.

4.1 Использование ядер на практике

Выбор ядра — это выбор класса решающих функций $f(x) = \sum_i \lambda_i y^{(i)} K(x^{(i)}, x) + b$, поэтому он влияет и на качество модели, и на скорость обучения и предсказания. Ниже собраны рекомендации, как этот выбор делать.

Какие ядра используют. Чаще всего встречаются четыре ядра (таблица 1).

Начинать стоит с линейного ядра: если оно уже дает приемлемое качество, усложнять модель незачем. Например, на цифрах `MNIST` («1» против «2») тестовая `accuracy` линейного SVM равна 0.986, а SVM с ядром `RBF` — 0.995, то есть выигрыш небольшой. На `Phoneme` разница уже существенна (0.7775 и 0.8555). Если о данных ничего не известно, разумный выбор по умолчанию — `RBF`: оно принимает значения от 0 до 1,

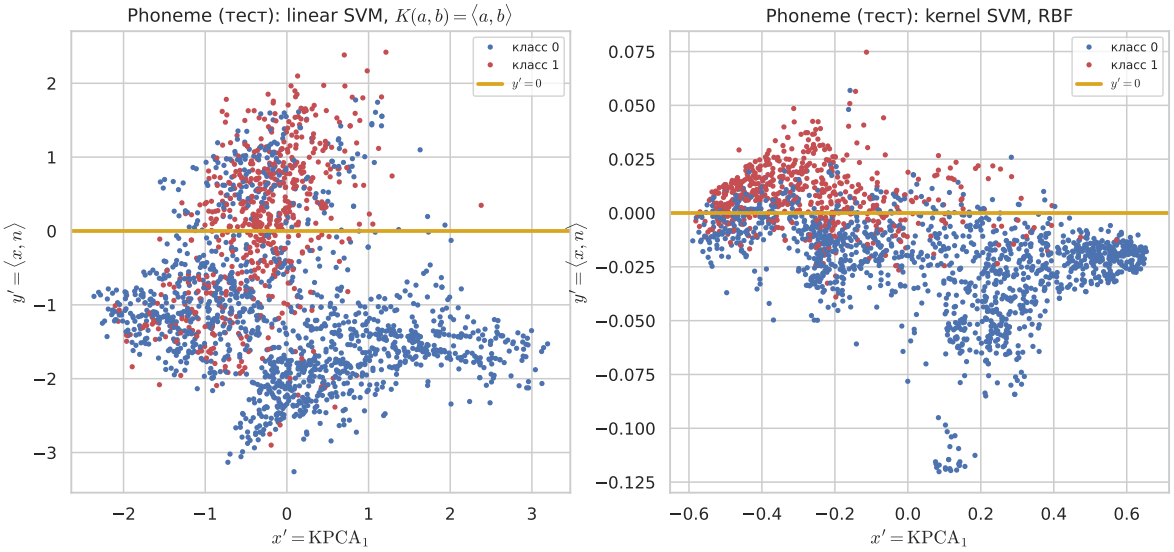


Рис. 6. Проекция (x', y') тестовой выборки `Phoneme`: линейный SVM (слева) и SVM с ядром `RBF` (справа).

Таблица 1. Распространенные ядра и их параметры в `SVC`.

Ядро	$K(x,z)$	Параметры
Линейное	$\langle x,z\rangle$	—
Полиномиальное	$\big(\gamma\langle x,z\rangle + r\big)^d$	`degree` d, `gamma` γ, `coef0` r
Гауссовское (RBF)	$\exp\big(-\gamma\|x-z\|^2\big)$	`gamma` γ
Сигмоидное	$\tanh\big(\gamma\langle x,z\rangle + r\big)$	`gamma` γ, `coef0` r

равно 1 при $x = z$ и убывает с расстоянием, то есть служит мерой сходства объектов. Сигмоидное ядро используют редко, так как при некоторых значениях параметров оно не является неотрицательно определенным.

Параметр γ ядра RBF. Параметр γ задает масштаб расстояний, на котором объекты считаются похожими. При малых γ ядро почти не меняется на масштабе выборки, и модель близка к линейной. При больших γ значение $K(x,z)$ быстро падает до нуля, и граница «облегает» отдельные объекты: точность на обучающей выборке стремится к 1, а на тестовой падает. В `scikit-learn` по умолчанию $\gamma = 1/(d\cdot\mathrm{Var}(X))$ (`gamma='scale'`); именно это значение использовано в примерах выше.

Масштаб признаков. Ядра `RBF` и полиномиальное зависят от расстояний и скалярных произведений, поэтому признак с большим масштабом будет доминировать в $\|x-z\|$. Перед обучением признаки нужно стандартизовать (в примере с `Phoneme` это сделано по статистикам обучающей выборки).

Подбор гиперпараметров. Параметры C и γ влияют на сложность модели согласованно: большие C и большие γ дают самую гибкую модель, поэтому подбирать

их нужно вместе, по кросс-валидации на логарифмической сетке (например, $C, \gamma \in \{10^{-3}, 10^{-2}, \ldots, 10^{3}\}$). Признаки переобучения — большой разрыв между качеством на обучающей и тестовой выборках (на `Phoneme` при $C = 100$ это 0.904 и 0.8555) и большая доля опорных векторов. Для сравнения: в примере с окружностями линейный SVM, для которого выборка неразделима, делает опорными все 1000 объектов, а SVM со стереографическим ядром — только 4.

Стоимость ядра. Обучение с ядром требует матрицы K_{ij} и работает заметно дольше линейных методов (раздел 5.2). Предсказание тоже дороже: чтобы вычислить $f(x)$, нужно хранить все опорные векторы и вычислить $K(x^{(i)}, x)$ для каждого из них, то есть $O(|SV| \cdot d)$ операций для `RBF`, тогда как линейная модель хранит готовый вектор w и тратит $O(d)$. Если выборка очень велика, а нелинейность все же нужна, ядро можно приблизить явным отображением в пространство умеренной размерности и обучить на нем линейный SVM стохастическим методом (раздел 5.4).

Собственные ядра. Если известна природа данных (окружности, последовательности, графы, тексты), ядро лучше строить из этих знаний, как мы сделали в примере со стереографическим ядром. Корректное ядро симметрично и неотрицательно определено (теорема Мерсера). Новые ядра можно получать из известных: сумма и произведение ядер, а также cK при $c > 0$, снова являются ядрами. В `scikit-learn` собственное ядро можно передать в `SVC` функцией от двух матриц объектов (так сделано для стереографического ядра в примере выше) или заранее вычислить матрицу K_{ij} и указать `kernel='precomputed'`.

5 Как обучают SVM на практике

Мы выписали несколько эквивалентных постановок задачи SVM, но пока не сказали, как их решать. Для бинарной классификации у нас есть три формы одной и той же задачи:

- **условная прямая задача** (5): квадратичная задача с $d+1+l$ переменными $(w,\, b,\, \xi)$ и $2l$ ограничениями;
- **безусловная задача с hinge loss** (7): всего $d+1$ переменная и никаких ограничений;
- **двойственная задача**: l переменных λ_i, ограничения $0 \leqslant \lambda_i \leqslant C$ и одно равенство $\sum_i \lambda_i y^{(i)} = 0$. Поскольку объекты входят в нее только через скалярные произведения, их можно заменить ядром $K(x^{(i)}, x^{(j)})$.

Первую форму в исходном виде, с переменными ξ_i и $2l$ ограничениями, универсальными QP-решателями решают редко: в ней больше всего переменных и ограничений. Однако от прямой задачи на практике не отказываются. Исключив ξ_i, мы получаем вторую форму, и именно прямую задачу (в варианте с квадратом hinge loss) решает библиотека `liblinear`, на которой основан `LinearSVC`. Выбор постановки определяется главным образом тем, нужно ли ядро, а в линейном случае— размером выборки и числом признаков (таблица 2).

Таблица 2. Способы обучения SVM для бинарной классификации.

Постановка	Ядра	Метод	`scikit-learn`
Безусловная (hinge loss)	нет	SGD	`SGDClassifier`
Безусловная (квадрат hinge loss)	нет	метод Ньютона	`LinearSVC(dual`
Двойственная (линейный случай)	нет	покоординатный подъем	`LinearSVC(dual`
Двойственная (с ядром)	да	SMO	`SVC`
Безусловная по признакам $K(x, z_m)$	приближенно	SGD	`Nystroem` + `SGD`

5.1 Градиентный спуск по hinge loss

Для простоты будем считать, что в объекты добавлен константный признак, и сдвиг b стал одной из компонент w. Перепишем задачу (7), поделив ее на l и обозначив $\alpha = \frac{1}{Cl}$:

$$Q(w) = \frac{\alpha}{2}\|w\|^2 + \frac{1}{l}\sum_{i=1}^{l} \max\bigl(0, 1 - y^{(i)}\langle w, x^{(i)}\rangle\bigr) \longrightarrow \min_w .$$

Это сумма регуляризатора и средних по объектам потерь— ровно такой же вид, как у логистической регрессии, которую вы обучали градиентным спуском. Единственное препятствие— hinge loss имеет излом в точке $m = 1$ и в нем не дифференцируема.

Решение проблемы с изломом. Посмотрим на вклад одного объекта как на функцию от w:

- если $m^{(i)} = y^{(i)}\langle w, x^{(i)}\rangle > 1$ (объект на своей стороне с запасом), то потери равны нулю при всех близких w, и их градиент равен 0;
- если $m^{(i)} < 1$ (объект в полосе или на чужой стороне), то потери равны $1 - y^{(i)}\langle w, x^{(i)}\rangle$, это линейная функция от w, и ее градиент равен $-y^{(i)}x^{(i)}$.

Ровно в точке излома $m^{(i)} = 1$ можно взять любое из этих двух значений (или любое «промежуточное»): строгая теория (субградиентные методы) показывает, что метод все равно сходится. Кроме того, при вещественных данных точное равенство $m^{(i)} = 1$ встречается крайне редко. Таким образом, «градиент» потерь на одном объекте равен

$$g_i(w) = \begin{cases} -y^{(i)}x^{(i)}, & y^{(i)}\langle w, x^{(i)}\rangle < 1, \\ 0, & \text{иначе.} \end{cases}$$

Интерпретация шага. Шаг градиентного спуска по одному объекту $w \leftarrow w - \eta\big(\alpha w + g_i(w)\big)$ устроен очень просто:

- регуляризатор всегда немного «сжимает» w к нулю, то есть расширяет полосу;
- если объект нарушает полосу, к w добавляется $\eta\, y^{(i)}x^{(i)}$: гиперплоскость поворачивается так, чтобы отступ этого объекта вырос;
- если объект уже лежит за полосой на своей стороне, он не влияет на шаг. Это та же разреженность, что и в двойственной задаче: на решение влияют только объекты в полосе и на чужой стороне.

Длина шага. Из-за излома градиент не стремится к нулю при подходе к минимуму: даже рядом с решением он скачком меняется, когда объект пересекает границу полосы. Поэтому с постоянным шагом метод «прыгает» вокруг минимума, и шаг уменьшают со временем. Самый известный вариант — алгоритм Pegasos (Shalev-Shwartz et al., 2007) с шагом $\eta_t = \frac{1}{\alpha t}$.

Каждый шаг стоит $O(d)$ и не зависит от размера выборки, поэтому метод хорошо подходит для очень больших выборок (миллионы объектов). Плата за это — отсутствие ядер: мы работаем с вектором w в исходном пространстве признаков. Как обойти это ограничение, обсудим в разделе 5.4. В `scikit-learn` этот подход реализован в `SGDClassifier(loss='hinge')`. Заменив параметр `loss`, тем же классом можно обучать и логистическую регрессию.

Algorithm 1 Pegasos: стохастический градиентный спуск для линейного SVM

Require: выборка $(x^{(i)}, y^{(i)})_{i=1}^{l}$, коэффициент регуляризации α, число шагов T

$w \leftarrow 0$
for $t = 1, \ldots, T$ **do**
  выбрать случайный номер объекта i
  $\eta_t \leftarrow \frac{1}{\alpha t}$
  if $y^{(i)}\langle w, x^{(i)}\rangle < 1$ **then**
    $w \leftarrow (1 - \eta_t \alpha)\, w + \eta_t\, y^{(i)} x^{(i)}$ ▷ объект нарушает полосу
  else
    $w \leftarrow (1 - \eta_t \alpha)\, w$ ▷ только регуляризация
  end if
end for
return w

5.2 Решение двойственной задачи: SMO

Если нужно ядро, приходится решать двойственную задачу

$$\max_{\lambda} \quad \sum_{i=1}^{l} \lambda_i - \frac{1}{2} \sum_{i,j=1}^{l} \lambda_i \lambda_j y^{(i)} y^{(j)} K(x^{(i)}, x^{(j)})$$

$$\texttt{s.t.} \quad 0 \leqslant \lambda_i \leqslant C, \quad \sum_{i=1}^{l} \lambda_i y^{(i)} = 0.$$

Это задача квадратичного программирования, и для нее существуют универсальные методы. Однако им нужна вся матрица $K_{ij} = K(x^{(i)}, x^{(j)})$ размера $l \times l$. Уже при $l = 10^5$ это 10^{10} чисел, то есть около 80 ГБ памяти. Поэтому на практике используют специализированный метод SMO (Sequential Minimal Optimization, Platt, 1998).

Идея: менять по несколько переменных. Будем улучшать решение, меняя на каждом шаге лишь небольшую группу переменных λ_i и оставляя остальные фиксированными. Это похоже на покоординатный спуск: по одной переменной задача становится одномерной, и ее легко решить точно.

Почему переменных две. Одну переменную менять нельзя: если изменить только λ_i, нарушится равенство $\sum_k \lambda_k y^{(k)} = 0$. Поэтому SMO меняет сразу *пару* λ_i, λ_j, причем согласованно, так чтобы сумма $\lambda_i y^{(i)} + \lambda_j y^{(j)}$ осталась прежней. Тогда новые значения пары лежат на отрезке прямой внутри квадрата $[0, C]^2$, и задача сводится к максимизации функции одной переменной — параболы на отрезке. Ее максимум находится по явной формуле: берем вершину параболы и, если она вышла за пределы отрезка, обрезаем до ближайшего конца. Получается, что каждый шаг SMO — это решение школьной задачи о максимуме параболы на отрезке.

Замечание 5.1: Явная формула шага

Обозначим через $E_k = f(x^{(k)}) - y^{(k)}$ текущую ошибку модели на объекте k и $\eta = K_{ii} + K_{jj} - 2K_{ij}$. Тогда (без вывода) новое значение

$$\lambda_j^{\text{new}} = \lambda_j + \frac{y^{(j)}(E_i - E_j)}{\eta},$$

обрезанное до допустимого отрезка, а λ_i меняется так, чтобы сохранить $\lambda_i y^{(i)} + \lambda_j y^{(j)}$.

Какую пару выбрать. Из условий ККТ мы знаем, как должно выглядеть решение: $\lambda_i = 0$ у объектов за полосой, $0 < \lambda_i < C$ у объектов на ее границе, $\lambda_i = C$ у объектов внутри полосы или на чужой стороне. SMO на каждом шаге выбирает пару объектов, которые нарушают эти условия сильнее всего, и «чинит» их. Когда все условия выполнены с заданной точностью (в `scikit-learn` это параметр `tol`), алгоритм останавливается.

Algorithm 2 SMO (идея)

Require: матрица ядра (вычисляется по мере необходимости), параметр C, точность ε

$\lambda \leftarrow 0$ ▷ допустимая начальная точка
while условия ККТ нарушены сильнее, чем на ε **do**
    выбрать пару (i, j), сильнее всего нарушающую условия ККТ
    найти максимум функционала по (λ_i, λ_j) при $\lambda_i y^{(i)} + \lambda_j y^{(j)} = \text{const}$
    обновить ошибки E_k для всех k
end while
вычислить b
return λ, b

Почему это работает быстро. На каждом шаге нужны только две строки матрицы ядра, $K_{i\cdot}$ и $K_{j\cdot}$, а не вся матрица. Часто используемые строки хранятся в кэше (параметр `cache_size` в `scikit-learn`). Кроме того, объекты, которые давно не меняются и почти наверняка имеют $\lambda_i = 0$, временно исключаются из рассмотрения (shrinking). Тем не менее число шагов растет с размером выборки, и на практике время обучения растет примерно от l^2 до l^3.

Как найти b. Для опорного вектора на границе полосы ($0 < \lambda_i < C$) выполнено $y^{(i)} f(x^{(i)}) = 1$, откуда

$$b = y^{(i)} - \sum_{j=1}^{l} \lambda_j y^{(j)} K(x^{(j)}, x^{(i)}).$$

Теоретически все такие объекты дают одно и то же b, но из-за приближенного решения значения немного различаются, поэтому их усредняют. Если таких объектов

нет, то b выбирается как середина отрезка допустимых значений (раздел 3.1, где $\langle w, x^{(i)} \rangle$ заменяется на $\sum_j \lambda_j y^{(j)} K(x^{(j)}, x^{(i)})$).

Где используется. SMO реализован в библиотеке `libsvm`, на которой основаны классы `SVC`, `NuSVC`, `SVR`, `NuSVR` и `OneClassSVM`. Двойственные задачи SVR (9) и одноклассового SVM (11) устроены так же, поэтому их решает тот же алгоритм.

5.3 Линейный случай: покоординатный подъем

Если ядро не нужно, двойственную задачу тоже выгодно решать, но немного иначе. Включим, как и выше, сдвиг b в вектор w через константный признак. Тогда условие $\partial \mathcal{L} / \partial b = 0$ пропадает, а вместе с ним и равенство $\sum_i \lambda_i y^{(i)} = 0$. Остаются только ограничения $0 \leqslant \lambda_i \leqslant C$, и менять переменные можно по одной: каждый шаг — снова максимум параболы на отрезке $[0, C]$.

Главная экономия в том, что в линейном случае можно явно хранить вектор $w = \sum_i \lambda_i y^{(i)} x^{(i)}$ и обновлять его после каждого шага. Тогда матрица ядра вообще не нужна, а один шаг стоит $O(d)$. Такой метод (Hsieh et al., 2008) реализован в библиотеке `liblinear`, на которой основан `LinearSVC`. По умолчанию `LinearSVC` использует квадрат hinge loss $\max(0, 1 - m)^2$, а сдвиг b в нем регуляризуется вместе с w, поэтому его ответ может немного отличаться от `SVC(kernel='linear')`.

Прямая задача в `liblinear`. Помимо двойственного, в `liblinear` есть решатель прямой задачи. Он решает безусловную задачу вида (7), но с квадратом hinge loss. В отличие от hinge loss, квадрат $\max(0, 1 - m)^2$ непрерывно дифференцируем, поэтому к нему применим метод Ньютона с доверительной областью (trust region Newton, TRON; Lin et al., 2008). Переменных в такой задаче $d + 1$, и каждая итерация требует нескольких проходов по выборке. Поэтому прямой решатель выгоден, когда объектов много больше, чем признаков ($l \gg d$), а двойственный — в обратной ситуации ($d \gg l$, например для текстов). В `LinearSVC` выбор задается параметром `dual`; при `dual='auto'` (значение по умолчанию в новых версиях `scikit-learn`) решатель выбирается автоматически по соотношению l и d. Для обычного hinge loss (`loss='hinge'`) доступен только двойственный решатель.

5.4 Ядра для больших выборок: явное ядровое отображение

Масштабируемость постановок. Безусловная постановка (7) — это сумма потерь по объектам и регуляризатор, то есть стандартная задача минимизации эмпирического риска. Поэтому к ней применим любой стохастический метод оптимизации: SGD и Pegasos, а также Adam, SAG/SAGA и другие. Шаг по мини-батчу не зависит от размера выборки, данные можно подавать потоком, и выборка может даже не помещаться в память. Тем самым метод переносится на выборки произвольного

объема. Постановки в виде задач квадратичного программирования так не масштабируются. В двойственной задаче l переменных, связанных через матрицу ядра размера $l \times l$, и время работы SMO растет от l^2 до l^3.

Однако ядра мы до сих пор умели использовать только в двойственной задаче. Можно ли совместить ядро со стохастической оптимизацией?

> **Замечание 5.2**
>
> Формально ядро можно использовать и в прямой задаче. По теореме о представителе решение имеет вид $f(x) = \sum_{i=1}^{l} \beta_i K(x^{(i)}, x) + b$, и hinge loss можно минимизировать прямо по β (так устроен ядровой вариант Pegasos). Но тогда для вычисления $f(x)$ на каждом шаге нужна сумма по всем накопленным объектам, и стоимость шага растет вместе с выборкой.

Явное ядровое отображение. Практичный ответ дает следующий прием (explicit kernel mapping). Выберем M опорных точек $z_1, \ldots, z_M$ и заменим признаковое описание объекта на вектор его сходств с этими точками:

$$\varphi(x) = \big(K(x, z_1), \ldots, K(x, z_M)\big) \in \mathbb{R}^M.$$

На новых признаках обучим обычный линейный SVM с hinge loss любым стохастическим методом. Получим решающую функцию

$$f(x) = \langle \beta, \varphi(x) \rangle + b = \sum_{m=1}^{M} \beta_m K(x, z_m) + b.$$

Она имеет тот же вид, что и у ядрового SVM, только роль опорных векторов играют заранее выбранные точки z_m.

Точки z_m обычно выбирают одним из способов:

- вся обучающая выборка ($M = l$). Это в точности форма решения из теоремы о представителе, но признаков столько же, сколько объектов, поэтому вариант подходит только для небольших выборок;
- случайная подвыборка размера $M \ll l$;
- центры кластеров, найденные, например, методом k-средних. Они покрывают выборку равномернее, чем случайные точки, при том же M.

Вычисление признаков стоит $O(Md)$ на объект (для `RBF`), шаг SGD — $O(M)$, и предсказание тоже не зависит от l. Число M (обычно сотни или тысячи) становится гиперпараметром: компромиссом между точностью приближения ядра и скоростью.

Связь с методом Нюстрёма. У ядрового SVM регуляризатор равен $\|w\|^2 = \beta^\top K_{ZZ} \beta$, где $(K_{ZZ})_{mk} = K(z_m, z_k)$, а у линейной модели на признаках $\varphi(x)$ он равен

$\|\beta\|^2$. Чтобы задачи совпали, признаки домножают на $K_{ZZ}^{-1/2}$:

$$\tilde{\varphi}(x) = K_{ZZ}^{-1/2}\varphi(x), \qquad \langle\tilde{\varphi}(x), \tilde{\varphi}(x')\rangle = \varphi(x)^\top K_{ZZ}^{-1}\varphi(x') \approx K(x, x').$$

Это и есть метод Нюстрёма (класс `Nystroem` в `scikit-learn`): для $x, x' \in \{z_1, \ldots, z_M\}$ приближение точное. Альтернатива, не требующая опорных точек,— случайные признаки Фурье (`RBFSampler`), приближающие ядро `RBF`. В `scikit-learn` весь подход записывается конвейером `make_pipeline(StandardScaler(), Nystroem(n_components=M), SGDClassifier(loss='hinge'))`. Для потоковых данных у `SGDClassifier` есть метод `partial_fit`.

5.5 Калибровка вероятностей

SVM выдает оценку $f(x)$, которая определяет сторону и относительную удаленность объекта от разделяющей гиперплоскости, но сама по себе не является вероятностью. Чтобы получить вероятностные оценки, поверх $f(x)$ обучают логистическую функцию (шкалирование Платта) на дополнительных данных. В `SVC` это можно включить параметром `probability=True`. Для калибровки используется дополнительная кросс-валидация, поэтому обучение становится заметно медленнее.

6 Вариации Support Vector Machine

В предыдущих разделах мы разобрали применение SVM для решения задачи бинарной классификации (SVC). Идеи метода— максимизация отступа, мягкие ограничения со штрафом C и переход к двойственной задаче— переносятся и на другие постановки. В этом разделе мы кратко рассмотрим три таких обобщения.

6.1 Многоклассовый SVC

Предположим, что у нас есть задача классификации на K классов и $K > 2$. Метки принимают значения $y^{(i)} \in \{1, \ldots, K\}$. Поскольку метод опорных векторов по построению бинарный, его можно обобщить на многоклассовую задачу стандартными способами, рассмотренными на прошлых занятиях:

- **один против всех** (one-vs-rest): обучается K классификаторов, каждый отделяет один класс от остальных; объекту присваивается класс, для которого значение $\langle w_k, x\rangle + b_k$ максимально;
- **каждый с каждым** (one-vs-one): обучается $K(K-1)/2$ классификаторов по одному на пару классов; итоговый класс выбирается голосованием.

Помимо двух перечисленных выше способов обобщения, существует более концептуальный способ обобщить именно SVM на многие классы.

Для упрощения выкладок, чтобы каждый раз не указывать отдельно b, заранее договоримся, что у нас имеется константный признак в выборке.

Вместо независимого обучения K классификаторов (как в one-vs-*) все K наборов весов $w_1, \ldots, w_K$ можно настраивать **совместно** в рамках одной оптимизационной задачи. Итоговый классификатор выглядит следующим образом:

$$a(x) = \arg \max_{k \in \{1,\ldots,K\}} \langle w_k, x \rangle.$$

Введем следующую функцию потерь:

$$\max_k \Big(\langle w_k, x \rangle + 1 - [k = y(x)] \Big) - \langle w_{y(x)}, x \rangle.$$

- Если $k = y(x)$, то выражение внутри максимума равно $\langle w_k, x \rangle$.
- Если $k \neq y(x)$, то оно равно $\langle w_k, x \rangle + 1$.

Если оценка правильного класса хотя бы на 1 больше оценок всех остальных классов, максимум достигается при $k = y(x)$ и потери равны нулю. Иначе — положительны. Это прямая аналогия с бинарным SVM: штрафуется не только ошибка, но и «неуверенная» классификация (объект попадает в разделяющую полосу).

Аналогом отступа в таком случае служит норма Фробениуса матрицы W, строки которой — векторы w_k:

$$\rho = \frac{1}{\|W\|_F^2} = \frac{1}{\sum_{k=1}^{K} \sum_{j=1}^{d} w_{kj}^2}.$$

Таким образом, приходим к задаче:

$$\begin{cases} \dfrac{1}{2}\|W\|_F^2 \to \min_W, \\ \langle w_{y_i}, x_i \rangle + [y_i = k] - \langle w_k, x_i \rangle \geqslant 1, \quad i = 1, \ldots, \ell; \ k = 1, \ldots, K. \end{cases}$$

Для случаев, когда невозможно свести потери к нулю, по аналогии с бинарным SVM, вводятся неотрицательные штрафы ξ_i за ошибки и неуверенную классификацию:

$$\begin{cases} \dfrac{1}{2}\|W\|_F^2 + C \sum_{i=1}^{\ell} \xi_i \to \min_{W,\xi}, \\ \langle w_{y_i}, x_i \rangle + [y_i = k] - \langle w_k, x_i \rangle \geqslant 1 - \xi_i, \\ \xi_i \geqslant 0. \end{cases}$$

В отличие от one-vs-all, все классификаторы обучаются одновременно, поэтому выдаваемые ими оценки автоматически согласованы друг с другом (нет проблемы разных масштабов, которая возникает при независимом обучении K бинарных SVM).

6.1.1 Реализация

- **«Каждый с каждым» и «один против всех»** не требуют ничего нового: задача сводится к набору бинарных SVM, каждый из которых обучается методами раздела 5. Ядровой `SVC` (`libsvm`, SMO) использует схему «каждый с каждым»: каждая из $K(K-1)/2$ задач содержит объекты только двух классов, поэтому они маленькие, а время SMO растет быстрее, чем линейно, по размеру выборки. Линейный `LinearSVC` (`liblinear`, покоординатный подъем) по умолчанию использует схему «один против всех».
- **Совместная постановка** (Crammer, Singer) реализована только для линейного случая: `LinearSVC(multi_class='crammer_singer')`. Ее двойственная задача решается тем же покоординатным подъемом, только на каждом шаге обновляются сразу K двойственных переменных, относящихся к одному объекту.

6.2 SVM для задач регрессии (SVR)

В задаче регрессии $y^{(i)} \in \mathbb{R}$, и нужно найти функцию $f(x) = \langle w, x \rangle + b$. Идея SVM сохраняется: ошибка, не превосходящая ε, не штрафуется, то есть вокруг гиперплоскости строится «трубка» ширины 2ε, а штрафуются только объекты вне нее.

Обратите внимание, что роль полосы здесь противоположна SVC (рисунок 7): в классификации объекты должны лежать *вне* полосы, и ее ширина $2/\|w\|$ максимизируется, а в регрессии объекты должны лежать *внутри* трубки, и ее ширина 2ε задается заранее как гиперпараметр. Минимизация $\frac{1}{2}\|w\|^2$ в SVR означает поиск максимально «плоской» функции.

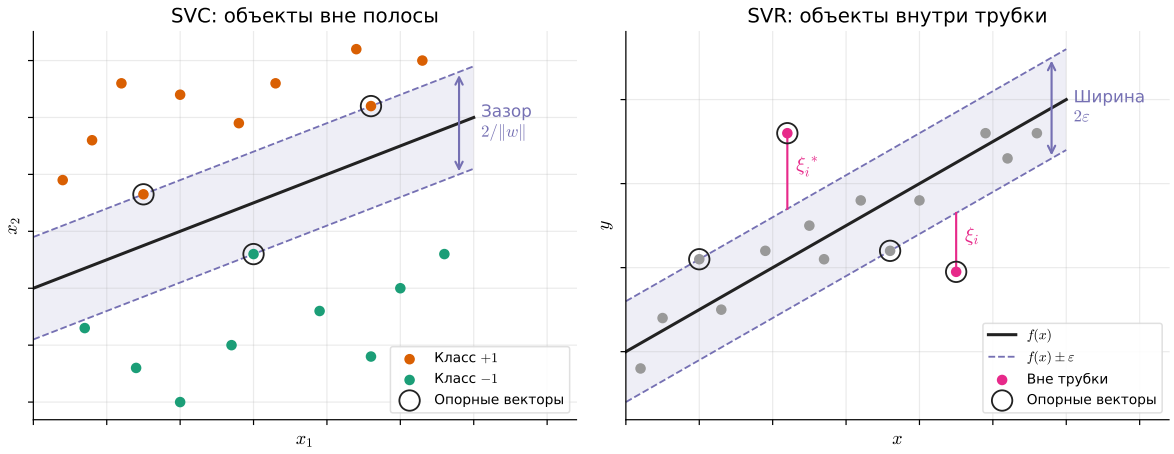


Рис. 7. В SVC объекты должны лежать вне полосы, в SVR — внутри трубки. Опорные векторы обведены.

6.2.1 ε-нечувствительная функция потерь

Определение 6.1: ε-нечувствительная функция потерь

Функция

$$|z|_\varepsilon := \max\bigl(0, |z| - \varepsilon\bigr), \qquad \varepsilon \geqslant 0,$$

называется **ε-нечувствительной функцией потерь** (epsilon-insensitive loss).

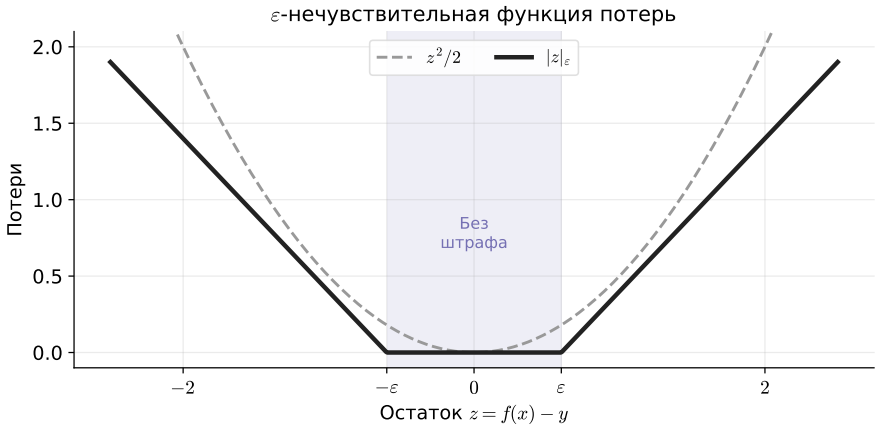


Рис. 8. ε-нечувствительная функция потерь (черная) и квадратичная функция потерь (пунктир).

Ошибка на объекте $x^{(i)}$ равна $|f(x^{(i)}) - y^{(i)}|_\varepsilon$ (рисунок 8): она нулевая, если объект попал внутрь трубки, и растет линейно по мере удаления от нее. Это прямой аналог hinge loss в SVC (раздел 3.2): объекты, «достаточно хорошо» описанные моделью, не влияют на решение. Кроме того, из-за линейного (а не квадратичного, как в MSE) роста потерь отдельные выбросы смещают решение слабее.

Задача обучения SVR записывается как

$$\frac{1}{2}\|w\|^2 + C\sum_{i=1}^{l}\bigl|\langle w, x^{(i)}\rangle + b - y^{(i)}\bigr|_\varepsilon \longrightarrow \min_{w,b}.$$

Слагаемое $\frac{1}{2}\|w\|^2$ отвечает за «плоскость» функции f (регуляризацию), параметр C задает баланс между ней и ошибками на обучающей выборке.

6.2.2 Прямая задача

Как и в разделе про неразделимый случай, избавимся от негладкой функции потерь, введя вспомогательные переменные. Отличие в том, что объект может выйти из трубки в две стороны: оказаться выше нее или ниже. Поэтому вводится два набора переменных: $\xi_i \geqslant 0$— величина выхода объекта за нижнюю границу трубки (предсказание завышено) и $\xi_i^* \geqslant 0$— за верхнюю (предсказание занижено). Получаем задачу

квадратичного программирования:

$$\begin{aligned} & \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{l}(\xi_i + \xi_i^*) \longrightarrow \min_{w,b,\xi,\xi^*}, \\ \texttt{s.t.} \quad & \langle w, x^{(i)}\rangle + b - y^{(i)} \leqslant \varepsilon + \xi_i, \quad i = 1,\ldots,l, \\ & y^{(i)} - \langle w, x^{(i)}\rangle - b \leqslant \varepsilon + \xi_i^*, \quad i = 1,\ldots,l, \\ & \xi_i \geqslant 0, \quad \xi_i^* \geqslant 0, \quad i = 1,\ldots,l. \end{aligned}$$

Нетрудно видеть, что в оптимуме хотя бы одна из переменных ξ_i, ξ_i^* равна нулю, а $\xi_i + \xi_i^* = |f(x^{(i)}) - y^{(i)}|_\varepsilon$.

6.2.3 Двойственная задача

Введем множители Лагранжа $\lambda_i, \lambda_i^* \geqslant 0$ для первых двух групп ограничений и $\eta_i, \eta_i^* \geqslant 0$ для ограничений неотрицательности:

$$\begin{aligned} \mathcal{L} = \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{l}(\xi_i + \xi_i^*) - \sum_{i=1}^{l}\lambda_i\big(\varepsilon + \xi_i - \langle w, x^{(i)}\rangle - b + y^{(i)}\big) \\ - \sum_{i=1}^{l}\lambda_i^*\big(\varepsilon + \xi_i^* + \langle w, x^{(i)}\rangle + b - y^{(i)}\big) - \sum_{i=1}^{l}(\eta_i\xi_i + \eta_i^*\xi_i^*). \end{aligned}$$

Условия стационарности:

$$\begin{aligned} \nabla_w\mathcal{L} = 0 \quad &\Longrightarrow \quad w = \sum_{i=1}^{l}(\lambda_i^* - \lambda_i)x^{(i)}, \\ \frac{\partial\mathcal{L}}{\partial b} = 0 \quad &\Longrightarrow \quad \sum_{i=1}^{l}(\lambda_i^* - \lambda_i) = 0, \\ \frac{\partial\mathcal{L}}{\partial\xi_i} = 0, \quad \frac{\partial\mathcal{L}}{\partial\xi_i^*} = 0 \quad &\Longrightarrow \quad \eta_i = C - \lambda_i, \quad \eta_i^* = C - \lambda_i^*. \end{aligned} \tag{8}$$

Из последних равенств и неотрицательности η_i, η_i^* получаем $0 \leqslant \lambda_i, \lambda_i^* \leqslant C$. Как и в разделе 2, по b, ξ_i и ξ_i^* лагранжиан линеен, поэтому эти равенства— не просто условия экстремума, а условия конечности двойственной функции. При их выполнении слагаемые с ξ_i, ξ_i^* и b сокращаются, и аналогично выкладкам раздела 2 получаем двойственную задачу:

$$\begin{aligned} \max_{\lambda,\lambda^*} \quad & -\frac{1}{2}\sum_{i,j=1}^{l}(\lambda_i^* - \lambda_i)(\lambda_j^* - \lambda_j)\langle x^{(i)}, x^{(j)}\rangle - \varepsilon\sum_{i=1}^{l}(\lambda_i^* + \lambda_i) + \sum_{i=1}^{l}y^{(i)}(\lambda_i^* - \lambda_i) \\ \texttt{s.t.} \quad & 0 \leqslant \lambda_i, \lambda_i^* \leqslant C, \quad i = 1,\ldots,l, \\ & \sum_{i=1}^{l}(\lambda_i^* - \lambda_i) = 0. \end{aligned} \tag{9}$$

Подставив (8) в $f(x)$, получаем итоговую модель

$$f(x) = \sum_{i=1}^{l} (\lambda_i^* - \lambda_i)\langle x^{(i)}, x\rangle + b.$$

Как и в SVC, объекты входят в двойственную задачу и в ответ только через скалярные произведения, поэтому их можно заменить ядром $K(x^{(i)}, x)$ и получить нелинейную регрессию.

6.2.4 Опорные векторы

Условия дополняющей нежесткости имеют вид

$$\lambda_i\big(\varepsilon + \xi_i - f(x^{(i)}) + y^{(i)}\big) = 0, \qquad (C - \lambda_i)\,\xi_i = 0,$$
$$\lambda_i^*\big(\varepsilon + \xi_i^* + f(x^{(i)}) - y^{(i)}\big) = 0, \qquad (C - \lambda_i^*)\,\xi_i^* = 0.$$

При $\varepsilon > 0$ объект не может одновременно лежать на верхней и нижней границе трубки, поэтому $\lambda_i\lambda_i^* = 0$. Отсюда получаем три типа объектов:

- $|f(x^{(i)}) - y^{(i)}| < \varepsilon$ (строго внутри трубки): $\lambda_i = \lambda_i^* = 0$, объект не влияет на решение;
- $|f(x^{(i)}) - y^{(i)}| = \varepsilon$ (на границе трубки): $\xi_i = \xi_i^* = 0$, а ненулевым может быть один из множителей, $0 \leqslant \lambda_i \leqslant C$ или $0 \leqslant \lambda_i^* \leqslant C$;
- $|f(x^{(i)}) - y^{(i)}| > \varepsilon$ (вне трубки): $\lambda_i = C$ или $\lambda_i^* = C$.

Таким образом, **опорными векторами** в SVR являются объекты с $\lambda_i^* - \lambda_i \neq 0$; все они лежат на границе трубки или вне ее. Решение, как и в SVC, разреженно: чем больше ε, тем больше объектов попадает внутрь трубки и тем меньше опорных векторов.

Сдвиг b находится по любому опорному вектору на границе трубки. Если $0 < \lambda_i < C$, то $\xi_i = 0$ и $f(x^{(i)}) - y^{(i)} = \varepsilon$, откуда $b = y^{(i)} + \varepsilon - \langle w, x^{(i)}\rangle$. Аналогично, если $0 < \lambda_i^* < C$, то $b = y^{(i)} - \varepsilon - \langle w, x^{(i)}\rangle$. На практике для численной устойчивости значения b, полученные по разным опорным векторам, усредняют.

6.2.5 Сравнение с линейной регрессией

Без ядра SVR подбирает такую же линейную функцию $f(x) = \langle w, x\rangle + b$, как и обычная линейная регрессия. Сравним задачи гребневой регрессии (ridge) и SVR:

$$\sum_{i=1}^{l}\big(f(x^{(i)}) - y^{(i)}\big)^2 + \alpha\|w\|^2 \longrightarrow \min_{w,b}, \qquad C\sum_{i=1}^{l}\big|f(x^{(i)}) - y^{(i)}\big|_\varepsilon + \frac{1}{2}\|w\|^2 \longrightarrow \min_{w,b}.$$

Регуляризация одинакова, различаются только функции потерь (рисунок 8). Отсюда следуют преимущества SVR:

- **устойчивость к выбросам в** y: квадратичные потери растут как z^2, и один объект с большим остатком сильно смещает решение. У $|z|_\varepsilon$ рост линейный, а вклад каждого объекта в решение ограничен, так как $\lambda_i, \lambda_i^* \leqslant C$;
- **нечувствительность к мелкому шуму**: остатки, меньшие ε, не влияют на решение;
- **разреженность**: решение зависит только от опорных векторов. Для линейной модели это неважно, но в ядровом случае позволяет хранить лишь часть выборки, тогда как ядровая гребневая регрессия (kernel ridge) использует для предсказания все l объектов.

У SVR есть и недостатки:

- нет аналитического решения, как у $w = (X^\top X + \alpha I)^{-1} X^\top y$; приходится решать задачу квадратичного программирования, что для ядрового SVR на больших выборках долго;
- два гиперпараметра (C и ε) вместо одного, причем ε зависит от масштаба y;
- нет вероятностной интерпретации: метод наименьших квадратов является оценкой максимального правдоподобия при гауссовском шуме, что дает доверительные интервалы для коэффициентов. SVR же при $\varepsilon \to 0$ оценивает скорее условную медиану, а не условное среднее;
- при гауссовском шуме без выбросов гребневая регрессия обычно работает не хуже.

Таким образом, линейный SVR оправдан при наличии выбросов или когда допустима погрешность порядка ε, а основная сила метода проявляется в ядровом варианте.

6.2.6 Гиперпараметры

У SVR два основных гиперпараметра:

- ε— ширина нечувствительной трубки. Определяет, какие отклонения считаются допустимыми, и тем самым число опорных векторов. Значение ε стоит выбирать с учетом масштаба целевой переменной и уровня шума в ней;
- C— штраф за выход из трубки. При больших C модель стремится описать все объекты (риск переобучения), при малых — получается более «плоская» функция.

Вместо ε можно задавать параметр $\nu \in (0, 1]$ (`NuSVR`): он ограничивает сверху долю объектов вне трубки и снизу долю опорных векторов, а ширина трубки ε подбирается автоматически. Это аналог ν-свойства одноклассового SVM (теорема 6.2).

6.2.7 Реализация

Все три подхода из раздела 5 переносятся на SVR почти без изменений.

- **С ядром: SMO.** Двойственная задача (9) имеет ту же структуру, что и у SVC: квадратичный функционал, ограничения $0 \leqslant \lambda_i, \lambda_i^* \leqslant C$ и одно линейное равенство. Отличие лишь в том, что переменных $2l$, а не l. Поэтому `SVR` и `NuSVR` решают ее тем же методом SMO из `libsvm` (в обозначениях документации $\alpha_i \equiv \lambda_i$, $\alpha_i^* \equiv \lambda_i^*$).
- **Линейный случай: покоординатный подъем.** `LinearSVR` основан на `liblinear`: сдвиг включается в w, равенство в двойственной задаче пропадает, и переменные обновляются по одной. Параметр `loss` позволяет выбрать $|z|_\varepsilon$ или ее квадрат.
- **Градиентный спуск.** Безусловную задачу с $|z|_\varepsilon$ можно, как и в случае hinge loss, решать стохастическим градиентным спуском: «градиент» потерь на объекте равен 0 внутри трубки и $\pm x^{(i)}$ вне ее. Так работает `SGDRegressor(loss='epsilon_insensitive')`.

6.3 SVM для обнаружения выбросов (одноклассовый SVM)

6.3.1 Постановка задачи

Часто требуется понять, похож ли объект на «типичные» объекты выборки: обнаружить мошенническую транзакцию, неисправность оборудования по показаниям датчиков, сетевую атаку. Размеченных аномалий при этом обычно мало или нет вовсе, да и новые аномалии могут быть не похожи на уже известные. Поэтому задачу формулируют без учителя: дана выборка $x^{(1)}, \ldots, x^{(l)}$ без меток, и нужно построить функцию $f(x)$, которая положительна в области, где сосредоточена основная масса объектов, и отрицательна вне ее.

Различают две постановки:

- **обнаружение новизны** (novelty detection): обучающая выборка «чистая», то есть не содержит аномалий, и требуется определить, является ли аномальным *новый* объект;
- **обнаружение выбросов** (outlier detection): обучающая выборка сама загрязнена выбросами, и их нужно найти в ней же. Предполагается, что выбросов мало и они лежат в областях низкой плотности.

6.3.2 Прямая задача

Одноклассовый SVM (Schölkopf et al., 2001) сразу формулируется в спрямляющем пространстве $\varphi(x)$, соответствующем ядру $K(x, x') = \langle \varphi(x), \varphi(x') \rangle$; зачем нужно ядро, объясним ниже. Идея: отделить образы обучающих объектов от начала координат гиперплоскостью $\langle w, \varphi(x) \rangle = \rho$, лежащей от начала координат как можно дальше. Расстояние от начала координат до гиперплоскости равно $\rho / \|w\|$. Как и в SVC,

разрешим части объектов нарушать ограничения, введя штрафы ξ_i:

$$\begin{aligned} & \frac{1}{2}\|w\|^2 + \frac{1}{\nu l}\sum_{i=1}^{l}\xi_i - \rho \longrightarrow \min_{w,\rho,\xi}, \\ \texttt{s.t.} \quad & \langle w, \varphi(x^{(i)})\rangle \geqslant \rho - \xi_i, \quad i = 1, \ldots, l, \\ & \xi_i \geqslant 0, \quad i = 1, \ldots, l. \end{aligned} \tag{10}$$

Здесь $\nu \in (0, 1]$ играет роль параметра C: при малом ν штраф за нарушения велик и почти все объекты оказываются по «нормальную» сторону гиперплоскости. Решающая функция имеет вид

$$a(x) = \operatorname{sgn}\big(\langle w, \varphi(x)\rangle - \rho\big),$$

где $+1$ означает «нормальный» объект, -1— выброс.

6.3.3 Двойственная задача

Введем множители Лагранжа $\lambda_i \geqslant 0$ и $\eta_i \geqslant 0$:

$$\mathcal{L} = \frac{1}{2}\|w\|^2 + \frac{1}{\nu l}\sum_{i=1}^{l}\xi_i - \rho - \sum_{i=1}^{l}\lambda_i\big(\langle w, \varphi(x^{(i)})\rangle - \rho + \xi_i\big) - \sum_{i=1}^{l}\eta_i\xi_i.$$

Условия стационарности:

$$w = \sum_{i=1}^{l}\lambda_i\varphi(x^{(i)}), \qquad \sum_{i=1}^{l}\lambda_i = 1, \qquad \lambda_i = \frac{1}{\nu l} - \eta_i \leqslant \frac{1}{\nu l}.$$

Подставляя их в лагранжиан, получаем двойственную задачу

$$\begin{aligned} \min_{\lambda} \quad & \frac{1}{2}\sum_{i,j=1}^{l}\lambda_i\lambda_j K(x^{(i)}, x^{(j)}) \\ \texttt{s.t.} \quad & 0 \leqslant \lambda_i \leqslant \frac{1}{\nu l}, \quad i = 1, \ldots, l, \\ & \sum_{i=1}^{l}\lambda_i = 1, \end{aligned} \tag{11}$$

и решающую функцию

$$a(x) = \operatorname{sgn}\Big(\sum_{i=1}^{l}\lambda_i K(x^{(i)}, x) - \rho\Big).$$

Порог ρ находится по любому объекту с $0 < \lambda_i < \frac{1}{\nu l}$: для него $\xi_i = 0$ и ограничение выполнено как равенство, поэтому $\rho = \sum_j \lambda_j K(x^{(j)}, x^{(i)})$.

Условия дополняющей нежесткости $\lambda_i\big(\langle w, \varphi(x^{(i)})\rangle - \rho + \xi_i\big) = 0$ и $\big(\frac{1}{\nu l} - \lambda_i\big)\xi_i = 0$

снова дают разбиение объектов на три группы: строго внутри «нормальной» области ($\lambda_i = 0$), на границе ($0 \leqslant \lambda_i \leqslant \frac{1}{\nu l}$) и выбросы с $\xi_i > 0$ ($\lambda_i = \frac{1}{\nu l}$). Опорными векторами являются объекты с $\lambda_i > 0$.

6.3.4 Смысл параметра ν

Теорема 6.2: ν-свойство

Пусть $\rho \neq 0$ в решении задачи (10). Тогда

1. доля обучающих объектов, признанных выбросами ($\xi_i > 0$), не превосходит ν;
2. доля опорных векторов не меньше ν.

Доказательство. Оба утверждения следуют из ограничений двойственной задачи $\sum_i \lambda_i = 1$ и $\lambda_i \leqslant \frac{1}{\nu l}$.

1. У каждого выброса $\lambda_i = \frac{1}{\nu l}$. Если выбросов m, то $1 = \sum_i \lambda_i \geqslant \frac{m}{\nu l}$, откуда $\frac{m}{l} \leqslant \nu$.
2. Каждый из s опорных векторов вносит в сумму не более $\frac{1}{\nu l}$, поэтому $1 = \sum_i \lambda_i \leqslant \frac{s}{\nu l}$, откуда $\frac{s}{l} \geqslant \nu$.

■

Таким образом, ν имеет понятный смысл: это оценка доли аномалий в выборке. Например, при $\nu = 0.05$ модель допустит, что не более 5% обучающих объектов окажутся вне «нормальной» области. Этим одноклассовый SVM выгодно отличается от SVC, где подобрать C «из смысла задачи» трудно.

6.3.5 Роль ядра

Без ядра метод почти бесполезен: линейная функция $\langle w, x\rangle - \rho$ просто отсекает полупространство, и результат зависит от того, где оказалось начало координат. Смысл появляется при использовании гауссовского (RBF) ядра $K(x, x') = \exp\bigl(-\gamma\|x - x'\|^2\bigr)$:

- так как $K(x, x) = 1$, все образы $\varphi(x)$ лежат на единичной сфере, а так как $K(x, x') > 0$, все они лежат в одном «октанте». Отделяя их от начала координат, гиперплоскость отсекает от сферы «шапочку», в которую попадает основная масса объектов;
- в исходном пространстве решающая функция $\sum_i \lambda_i \exp\bigl(-\gamma\|x - x^{(i)}\|^2\bigr) - \rho$ является суммой гауссовых «бугорков» с центрами в опорных векторах, то есть по сути ядерной оценкой плотности, у которой отсечен порог ρ. «Нормальная» область может быть невыпуклой и даже несвязной.

Параметр γ задает ширину бугорков: при больших γ граница плотно облегает обучающие объекты (переобучение), при малых она становится слишком гладкой.

Замечание 6.3

Близкий метод SVDD (Support Vector Data Description, Tax, Duin, 2004) ищет в спрямляющем пространстве шар минимального радиуса, содержащий большую часть объектов. Для ядер с $K(x,x) = \text{const}$, в частности для RBF, задачи SVDD и одноклассового SVM эквивалентны.

6.3.6 Реализация и особенности

- **С ядром: SMO.** Двойственная задача (11) снова имеет вид «квадратичный функционал, ограничения коробкой $0 \leqslant \lambda_i \leqslant \frac{1}{\nu l}$ и одно равенство $\sum_i \lambda_i = 1$». Поэтому `OneClassSVM(kernel, nu, gamma)` решает ее тем же методом SMO из `libsvm`: переменные меняются парами так, чтобы сумма $\lambda_i + \lambda_j$ сохранялась.
- **Большие выборки: градиентный спуск.** Аналогично SVC, прямую задачу (10) можно переписать без ограничений, с потерями $\max\bigl(0, \rho - \langle w, \varphi(x^{(i)})\rangle\bigr)$, и решать стохастическим градиентным спуском за линейное по l время. Так работает `SGDOneClassSVM`. Ядер у него нет, поэтому для нелинейной границы признаки предварительно преобразуют аппроксимацией ядра (`Nystroem`, раздел 5.4).

Метод `predict` возвращает $+1$ для «нормальных» объектов и -1 для выбросов, а `decision_function` возвращает значение $\sum_i \lambda_i K(x^{(i)}, x) - \rho$, по которому объекты можно ранжировать по степени аномальности.

Одноклассовый SVM чувствителен к выбросам в обучающей выборке: попав в нее, они растягивают «нормальную» область. Поэтому метод лучше всего подходит для обнаружения новизны, когда обучающая выборка чистая.